

1. Objective

1. To understand the architecture of **Long Short-Term Memory (LSTM)** networks.
2. To analyze how LSTMs overcome the **vanishing gradient problem** of vanilla RNNs.
3. Go through the **gating mechanisms (Forget Gate, Input Gate, and Output Gate)** of an LSTM from scratch.(not necessary to implement)
4. To compare the performance of **LSTM and Vanilla RNN** models for sequential data.
5. To apply LSTM networks for **multi-step time-series forecasting**.
6. Make the appropriate assumptions for implementing the task and state them clearly in the outputs in each task

2. Problem 1: Sentiment Analysis Using Manual LSTM Implementation

Goal

Build a **Many-to-One LSTM model** to classify the sentiment of short text sequences.

Task 1: Dataset Preparation

Use a real sentiment analysis dataset such as:

- **IMDB Movie Review Dataset**

Steps

1. Load the dataset and inspect the data structure.
 2. Select a **balanced subset** of the dataset (equal positive and negative samples).
 3. Perform **text preprocessing**:
 - Convert text to lowercase
 - Remove punctuation
 - Tokenize sentences
 4. Create vocabulary mappings:
 - word_to_idx
 - idx_to_word
 5. Add special tokens:
 - <PAD> for padding
 - <UNK> for unknown words
 6. Convert sentences into **numerical index sequences**.
 7. Apply **padding** so that all sequences have the same length.
 8. Split the dataset into:
 - **80% Training**
 - **20% Testing** using stratified sampling..
-

Task 2: Training and Evaluation

1. Use **Binary Cross-Entropy Loss** for sentiment classification.
 2. Train the model for **at least 20 epochs**.(upto the model training is completed)(verify it how? By yourself)
 3. Plot the **training loss curve**.
 4. Evaluate the model on the **test dataset**.
 5. Report:
 - Final **classification accuracy**
 - **Confusion matrix**
-

3. Problem 2: Multi-Step Time-Series Forecasting Using LSTM

Goal

Use LSTM to predict the **next five values of a time-series signal**.

Task 1: Dataset Generation

Generate a composite signal using:

$$y(t) = \sin(t) + 0.5\cos(2t) + \text{noise}$$

Steps:

1. Generate at least **500 time steps**.
2. Add **Gaussian noise** to simulate real-world data.
3. Plot the generated time series.

Task 2: Windowing

Create training samples using a sliding window.

Input sequence:

$$(x_{t-49}, x_{t-48}, \dots, x_t)$$

Target sequence:

$$(x_{t+1}, x_{t+2}, \dots, x_{t+5})$$

Therefore:

- Input length = **50**
- Prediction horizon = **5**

Task 3: Model Implementation

For this task, you may use **PyTorch** or **TensorFlow**.

Model requirements:

1. Multi-layer LSTM network
2. Hidden size ≥ 32
3. Fully connected output layer producing **5 predictions**
4. **take appropriate loss function of your choice**

Task 4: Comparison with Vanilla RNN (optional)

1. Use the **Vanilla RNN implemented in Lab-6**.
2. Train both models on the same dataset.
3. Compare the models based on:
 - Prediction accuracy
 - Training stability
 - Ability to capture long-term dependencies.
4. Plot:
 - Actual signal
 - RNN prediction
 - LSTM prediction

4. Expected Observations

1. **Vanishing Gradient Problem**
Vanilla RNN struggles with long sequences, while LSTM preserves gradients using the cell state.
2. **Model Performance**
LSTM performs significantly better than vanilla RNN for sequences longer than **20–30 timesteps**.

5. Inference and Conclusion

Students must clearly discuss:

1. The role of the **cell state** C_t as a constant error carousel that allows gradients to flow across time steps.
2. How the **forget gate** prevents uncontrolled growth of memory.
3. Why **LSTMs require more parameters and computation** than vanilla RNNs.
4. Real-world applications where LSTMs are still useful, such as:
 - Speech recognition
 - Financial forecasting
 - Biomedical signal processing
 - Sequential sensor data analysis.

6. Submission Instructions

Students must submit a report containing:

1. **Objective**
2. **Hypothesis**
3. **Experimental Description**
4. **Observations**
5. **Conclusion**